They translate to calls to `since` or `advance`. For example here we get the correct jump in the calendar reform:

```
Date.new(1582, 10, 4) + 1.day
# => Fri, 15 Oct 1582
```

15.1.6 Timestamps

The following methods return a `Time` object if possible, otherwise a `DateTime`. If set, they honor the user time zone.

15.1.6.1 `beginning_of_day`, `end_of_day`

The method `beginning_of_day` returns a timestamp at the beginning of the day (00:00:00):

```
date = Date.new(2010, 6, 7)
date.beginning_of_day # => Mon Jun 07 00:00:00 +0200 2010
```

The method `end_of_day` returns a timestamp at the end of the day (23:59:59):

```
date = Date.new(2010, 6, 7)
date.end_of_day # => Mon Jun 07 23:59:59 +0200 2010
```

`beginning_of_day` is aliased to `at_beginning_of_day`, `midnight`, `at_midnight`.

15.1.6.2 `beginning_of_hour`, `end_of_hour`

The method `beginning_of_hour` returns a timestamp at the beginning of the hour (hh:00:00):

```
date = DateTime.new(2010, 6, 7, 19, 55, 25)
date.beginning_of_hour # => Mon Jun 07 19:00:00 +0200 2010
```

The method `end_of_hour` returns a timestamp at the end of the hour (hh:59:59):

```
date = DateTime.new(2010, 6, 7, 19, 55, 25)
date.end_of_hour # => Mon Jun 07 19:59:59 +0200 2010
```

`beginning_of_hour` is aliased to `at_beginning_of_hour`.

15.1.6.3 `beginning_of_minute`, `end_of_minute`

The method `beginning_of_minute` returns a timestamp at the beginning of the minute (hh:mm:00):

```
date = DateTime.new(2010, 6, 7, 19, 55, 25)
date.beginning_of_minute # => Mon Jun 07 19:55:00 +0200 2010
```

The method `end_of_minute` returns a timestamp at the end of the minute (hh:mm:59):

```
date = DateTime.new(2010, 6, 7, 19, 55, 25)
date.end_of_minute # => Mon Jun 07 19:55:59 +0200 2010
```

`beginning_of_minute` is aliased to `at_beginning_of_minute`.

`beginning_of_hour`, `end_of_hour`, `beginning_of_minute` and `end_of_minute` are implemented for `Time` and `DateTime` but **not** `Date` as it does not make sense to request the beginning or end of an hour or minute on a `Date` instance.

15.1.6.4 `ago`, `since`

The method `ago` receives a number of seconds as argument and returns a timestamp those many seconds ago from midnight:

```
date = Date.current # => Fri, 11 Jun 2010
date.ago(1)         # => Thu, 10 Jun 2010 23:59:59 EDT -04:00
```

Similarly, `since` moves forward:

```
date = Date.current # => Fri, 11 Jun 2010
date.since(1)       # => Fri, 11 Jun 2010 00:00:01 EDT -04:00
```

15.1.7 Other Time Computations

15.2 Conversions

16 Extensions to `DateTime`

`DateTime` is not aware of DST rules and so some of these methods have edge cases when a DST change is going on. For example `seconds_since_midnight` might not return the real amount in such a day.